

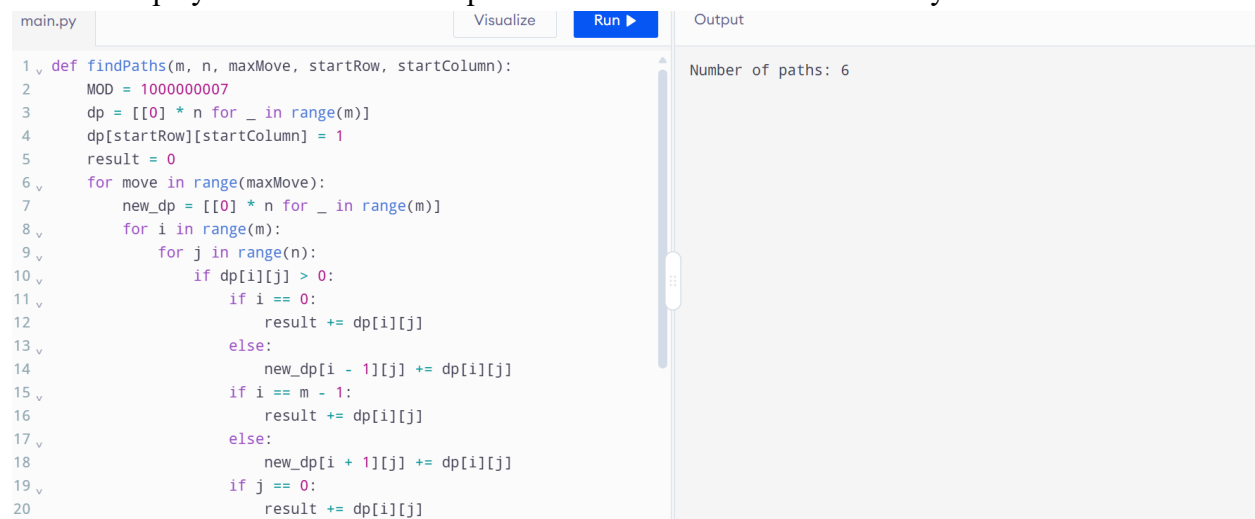
5.Out of Boundary Paths

Aim

To write a Python program using **Dynamic Programming** to find the number of paths by which a ball can move out of the boundary of a given grid within a specified number of moves.

Procedure

1. Create an $m \times n$ grid.
2. Store the starting position of the ball.
3. Use a DP table to store the number of ways the ball can reach each cell.
4. For every move, consider the four possible directions:
 - a. Up
 - b. Down
 - c. Left
 - d. Right
5. If a move goes outside the grid boundary, increase the result.
6. Otherwise, store the number of paths in the corresponding cell of the new DP table.
7. Repeat the process until the maximum number of moves is reached.
8. Display the total number of paths that move outside the boundary.



The screenshot shows a Python IDE with a file named 'main.py'. The code defines a function 'findPaths' that takes parameters 'm', 'n', 'maxMove', 'startRow', and 'startColumn'. It initializes a DP table 'dp' of size 'm' by 'n' with a modulus of 1000000007. The starting position is set at 'dp[startRow][startColumn] = 1'. A loop iterates over 'move' from 0 to 'maxMove'. For each move, a new DP table 'new_dp' is created. A nested loop iterates over 'i' from 0 to 'm-1' and 'j' from 0 to 'n-1'. If 'dp[i][j] > 0', it checks four directions: Up (i-1, j), Down (i+1, j), Left (i, j-1), and Right (i, j+1). If a move goes outside the grid boundary, the result is incremented. Otherwise, the number of paths is updated in 'new_dp'. The final result is displayed in the output window.

```
1 def findPaths(m, n, maxMove, startRow, startColumn):
2     MOD = 1000000007
3     dp = [[0] * n for _ in range(m)]
4     dp[startRow][startColumn] = 1
5     result = 0
6     for move in range(maxMove):
7         new_dp = [[0] * n for _ in range(m)]
8         for i in range(m):
9             for j in range(n):
10                if dp[i][j] > 0:
11                    if i == 0:
12                        result += dp[i][j]
13                    else:
14                        new_dp[i - 1][j] += dp[i][j]
15                if i == m - 1:
16                    result += dp[i][j]
17                else:
18                    new_dp[i + 1][j] += dp[i][j]
19                if j == 0:
20                    result += dp[i][j]
```

Output: Number of paths: 6

main.py		Visualize	Run ▶	Output
14	new_dp[i - 1][j] += dp[i][j]			Number of paths: 6
15	if i == m - 1:			
16	result += dp[i][j]			
17	else:			
18	new_dp[i + 1][j] += dp[i][j]			
19	if j == 0:			
20	result += dp[i][j]			
21	else:			
22	new_dp[i][j - 1] += dp[i][j]			
23	if j == n - 1:			
24	result += dp[i][j]			
25	else:			
26	new_dp[i][j + 1] += dp[i][j]			
27	dp = new_dp			
28	return result % MOD			
29	m = 2			
30	n = 2			
31	maxMove = 2			
32	startRow = 0			
33	startColumn = 0			
34	print("Number of paths:", findPaths(m, n, maxMove, startRow, startColumn))			

Result

Thus, the Python program successfully calculates the number of paths by which the ball can move **out of the boundary of the grid** within the given number of moves.